

Spatial AI Updates

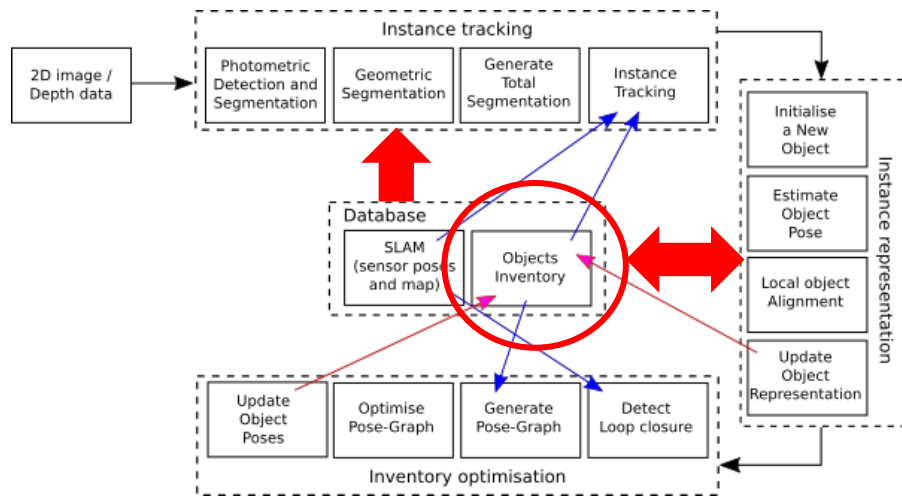
Outline

- Introduction.
- Architectures for implicit representations for SLAM.
- Implementations - Towards Object-SLAM with implicit shapes.
 - Demo: NeRF using pytorch3d.
 - Demo: DeepSDF.

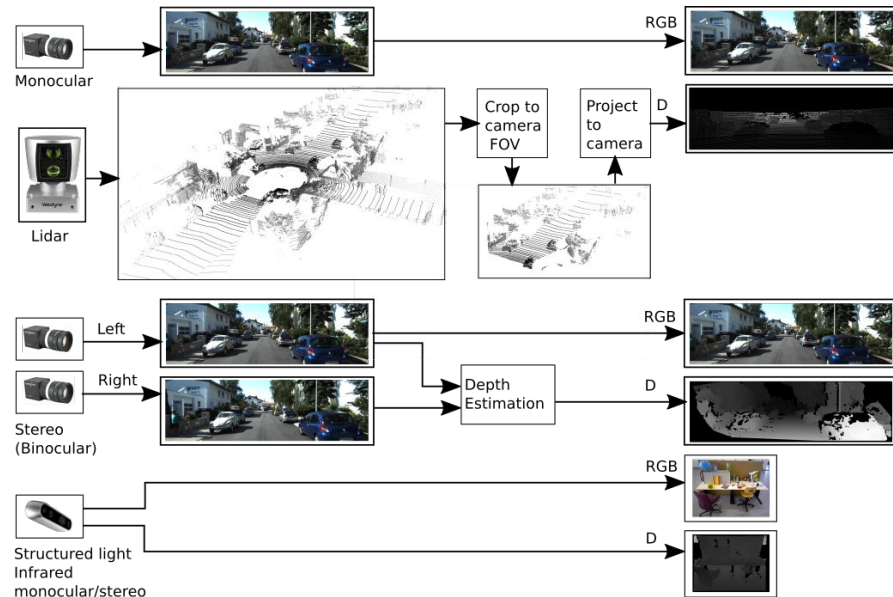
Introduction

Previous work

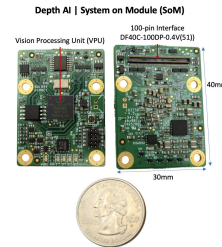
From SLAM to Spatial AI: Lines of attack



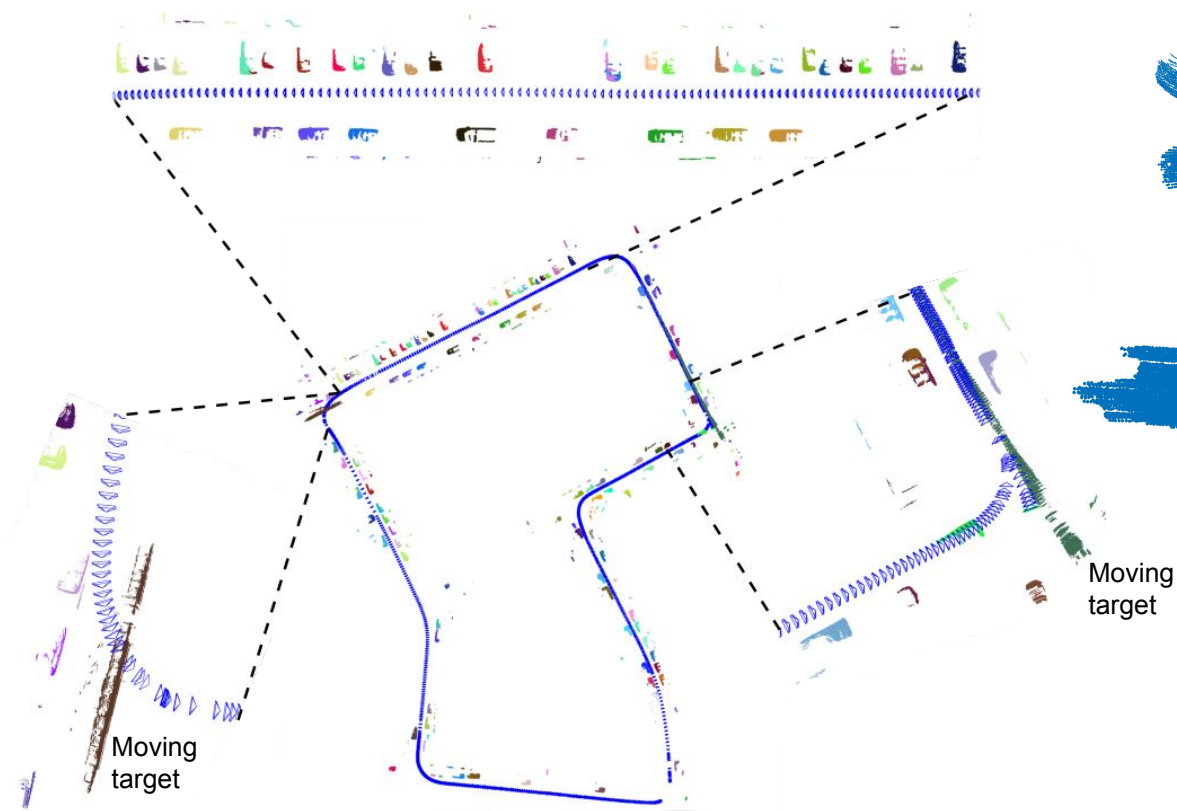
Object SLAM (Representations)



Hardware (sensors and processors)

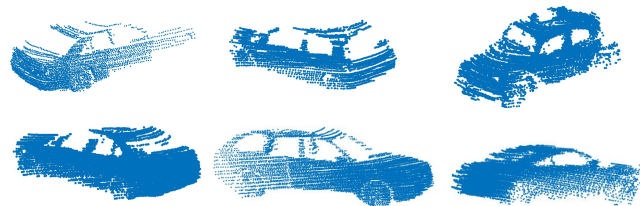


Previous work on Semantic SLAM

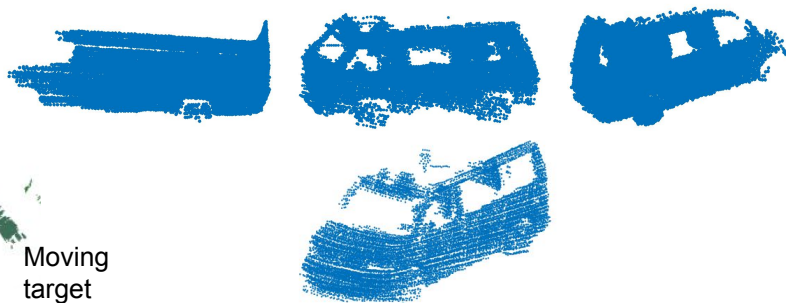


Inventory of objects (point-clouds)

Light vehicles



Heavy vehicles



Pedestrians



3D implicit representations

Rapidly growing body of work

Neural Implicit Surfaces

Occupancy networks

DeepSDF (regresses a TSDF)

Scene representation networks

Differential volumetric rendering

Implicit differential rendering

Real-time Systems

Fusion++ (Imperial College London)

MaskFusion

voxblox++

Explicit representations

Points

Meshes

Surfaces, Surfels

Voxels

TSDF

Future Mapping

Spatial AI Systems

(Imperial College London)

Semantic SLAM SIF, FY 2019-2020

Neural Volumes

Learning Dynamic Renderable

Volumes from Images (voxel-based)

(Facebook Reality Labs)

NodeSLAM

Real-time system

(Imperial College London)

NeRF

Regresses Density and color

Stored in an MLP

(Google research, UC Berkeley)

3D Dynamic

Scene Graphs

Hierarchical

Representations

(MIT)

pixelNeRF

Extension

(UC Berkeley)

iNeRF

Extension

(Google Research, MIT)

Object-Centric

Object scattering functions

(Google research)

Nerfies

Extension

(Google)

NeRF++

Performance

(Intel, Cornell)

Neural SDF

(Nvidia)

NSVF

Performance

(Facebook AI Research)

DONeRF

Performance

(Graz UT, Facebook Reality Labs)

iMAP

Real-time system

(Imperial College London)

NeRF in Detail

Performance

(Oxford Vision Lab)

BARF

Extension

(CMU, MIT, Adelaide)

Backing NeRF

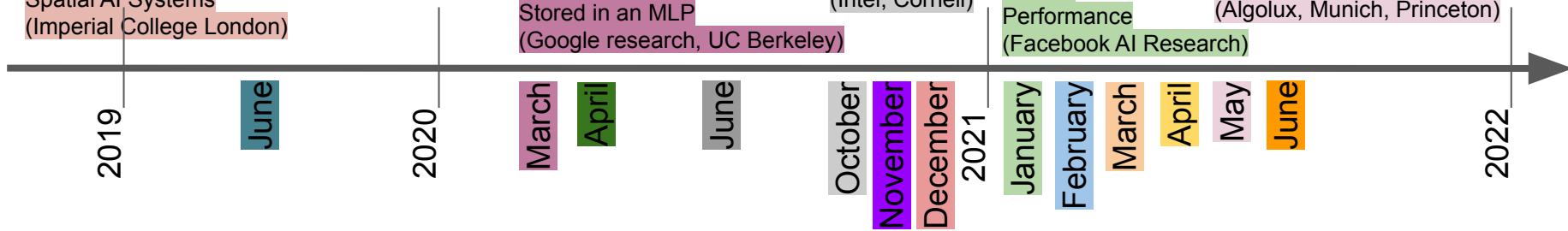
Performance

(Google research)

Neural Scene Graphs

Dynamic

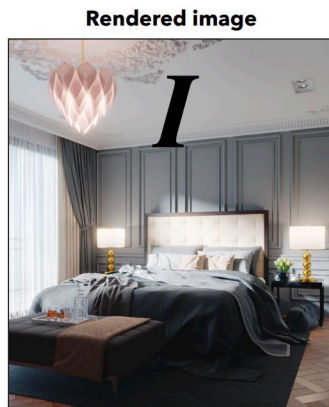
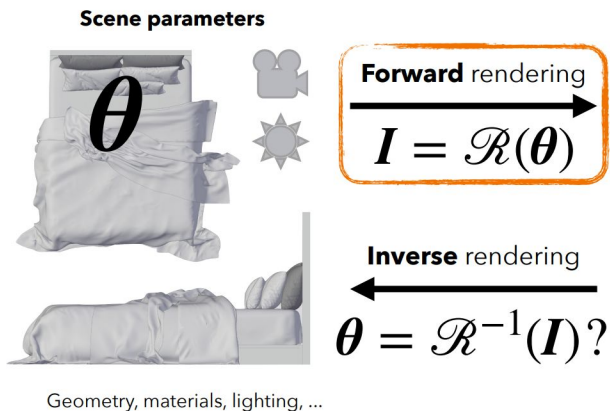
(Algolux, Munich, Princeton)



What is Differentiable Rendering? And Why bother?

Solving **inverse-rendering** (analysis by synthesis) problems.
i.e. inferring **scene parameters** based on **images** of the scene.

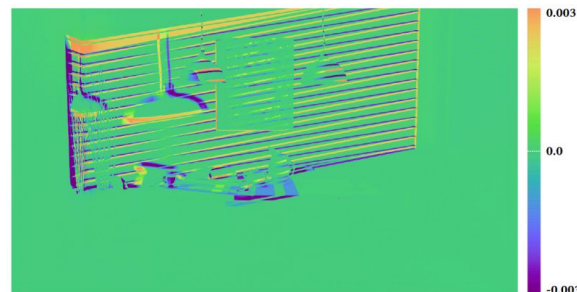
Integrating **forward-rendering** into **probabilistic inference** and **machine learning** pipelines
i.e., backpropagating losses during training.



Scene: "bed classic" from Jiriano

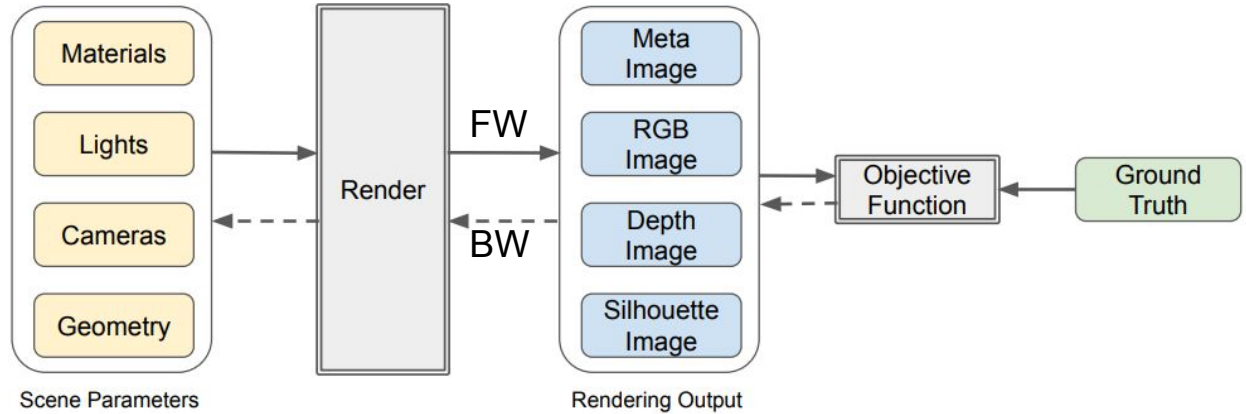


Forward-rendering results

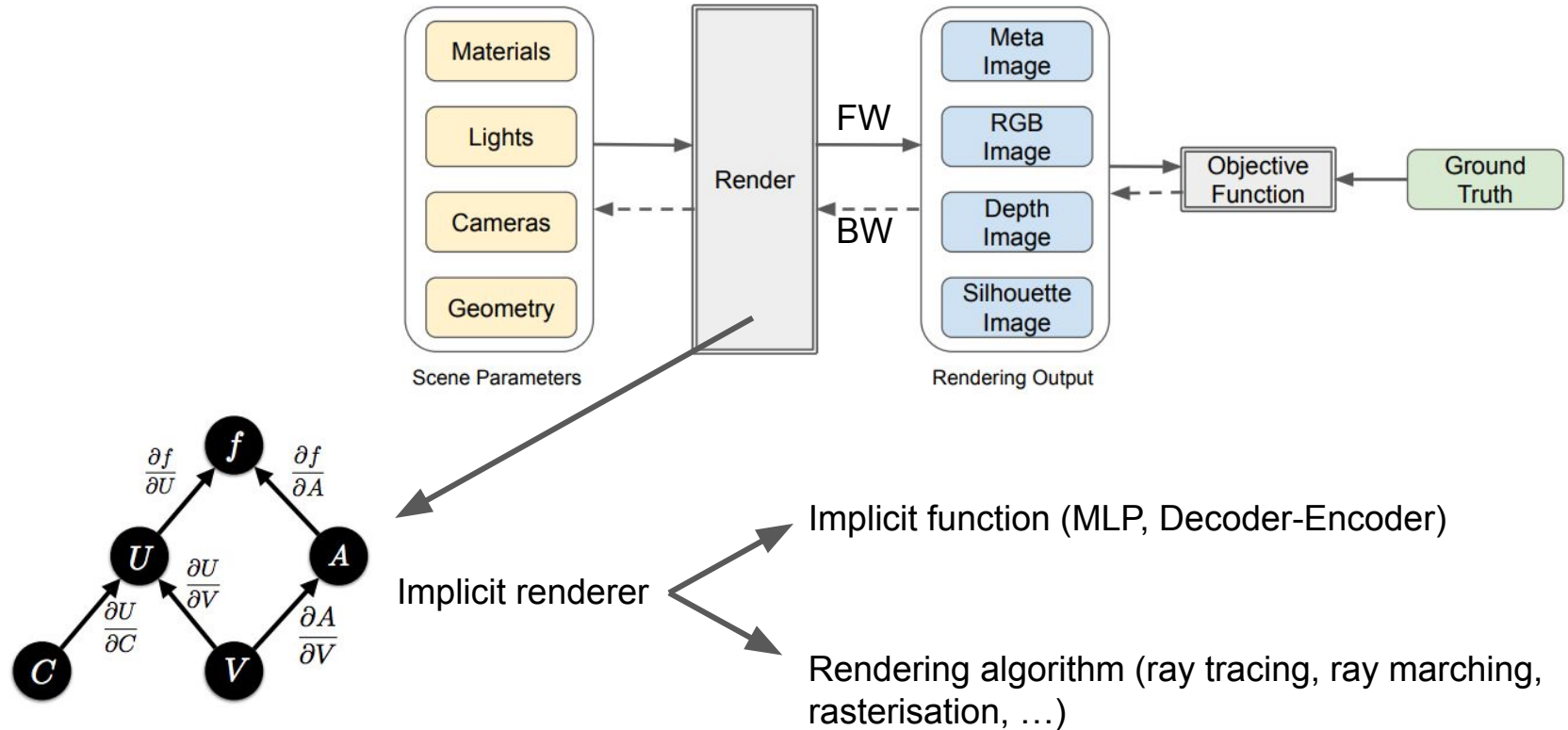


Differentiable-rendering result with respect to the location of the sun

Optimisation of Scene Parameters using a Renderer

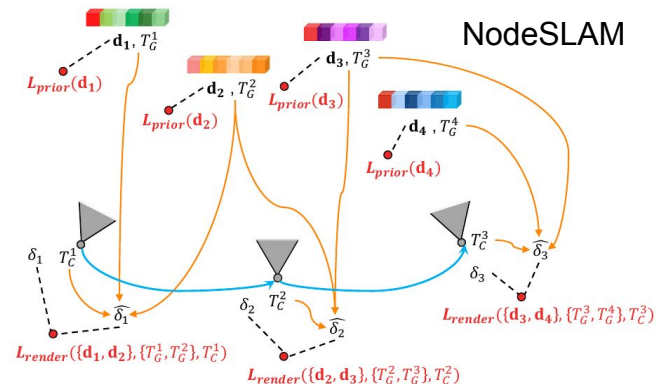


Optimisation of Scene Parameters using a Renderer

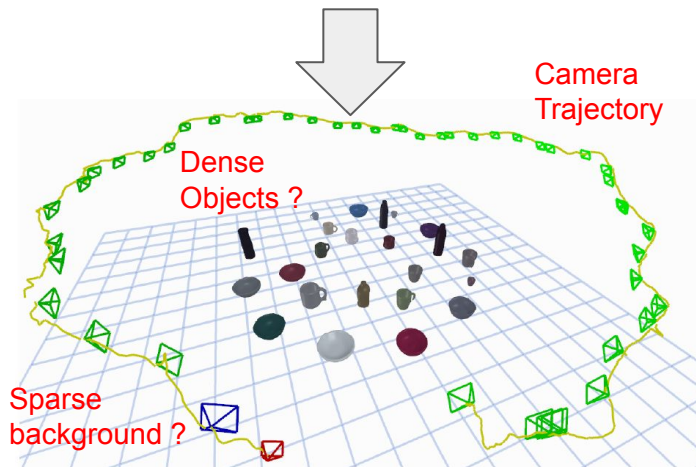


Why bother?

- Most of current autonomous robotic systems don't have the ability or capacity to utilise maps from SLAM, and thus to enable Spatial AI. Compact representations **reduce storage and bandwidth requirements of spatial AI systems**.
- It is desired for SLAM systems to capture semantic information to **enable high-level decision-making and spatial AI**.
- Prior knowledge of various objects in the map allows for **dense object reconstruction** from sparse sensor measurements. It also **increases the robustness of object tracking and 3D mapping** against noisy sensors data.



SLAM using a decoded object features



Collaborative SLAM problem description

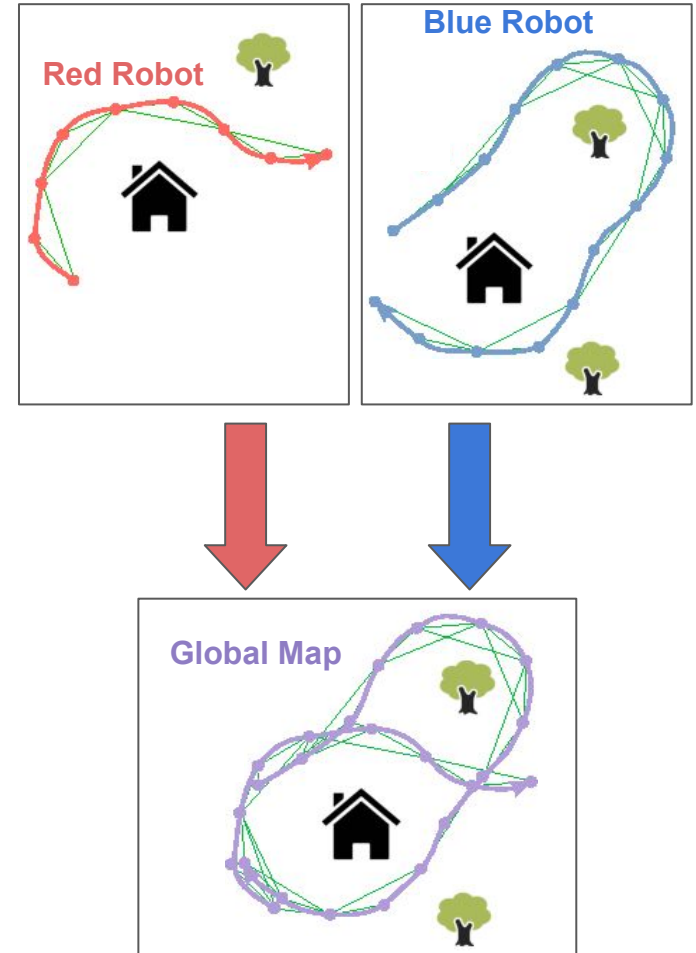
A number of agents are required to collaboratively generate a map of an environment **under limited communication bandwidth**.

Inter-robot loop closures are critical to align the trajectories of the robotics in a common reference frame and to improve trajectory estimates and remove duplicate objects in the map.

Agents are required to **share information on specific objects of interest** to enable high-level decision-making.

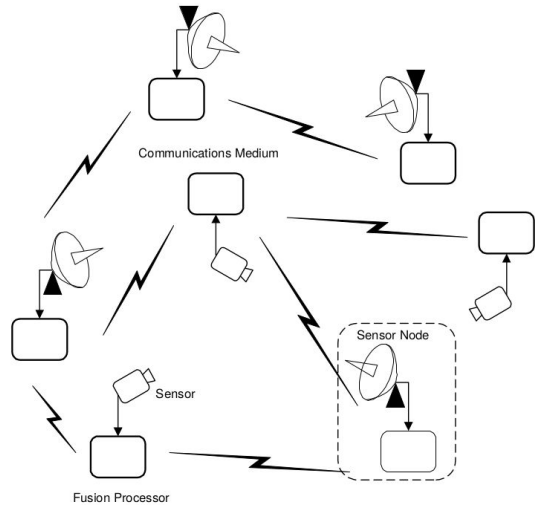
Agents are required to **maintain a map of the environment** for some objective critical tasks including pose-tracking, path planning and obstacle avoidance.

The agents should operate in a **decentralised fashion**. Hence, There isn't a central fusion unit. Each agent will maintain its own knowledge of the environment and its objects and carry on its object-level SLAM tasks including object-detection and classification, visual odometry, pose-graph optimisation, and local map optimisation.

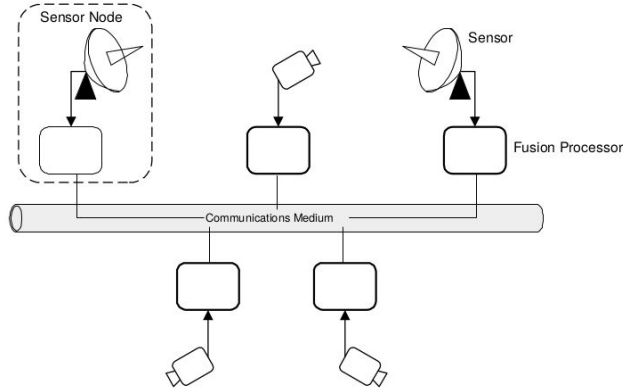


Decentralised Data Fusion

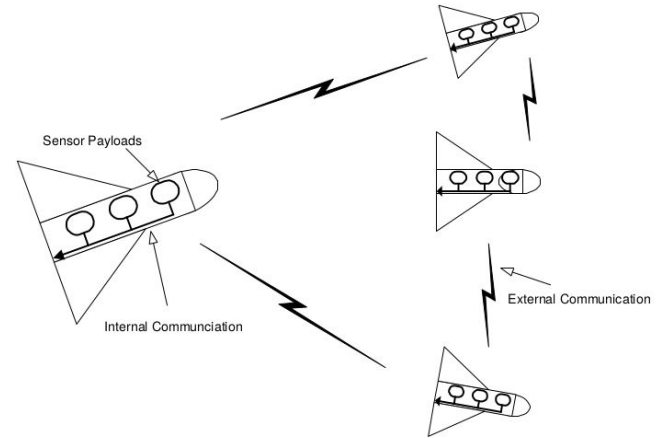
- A **restricted** network of sensor nodes, characterised by:
 - No single central fusion unit.
 - No common communication facility (no broadcasting of results, strictly node-to-node communication).
 - No global knowledge of sensor network topology (only connections in own neighbourhood).
- Possible realisations:



Point-to-point communication architecture

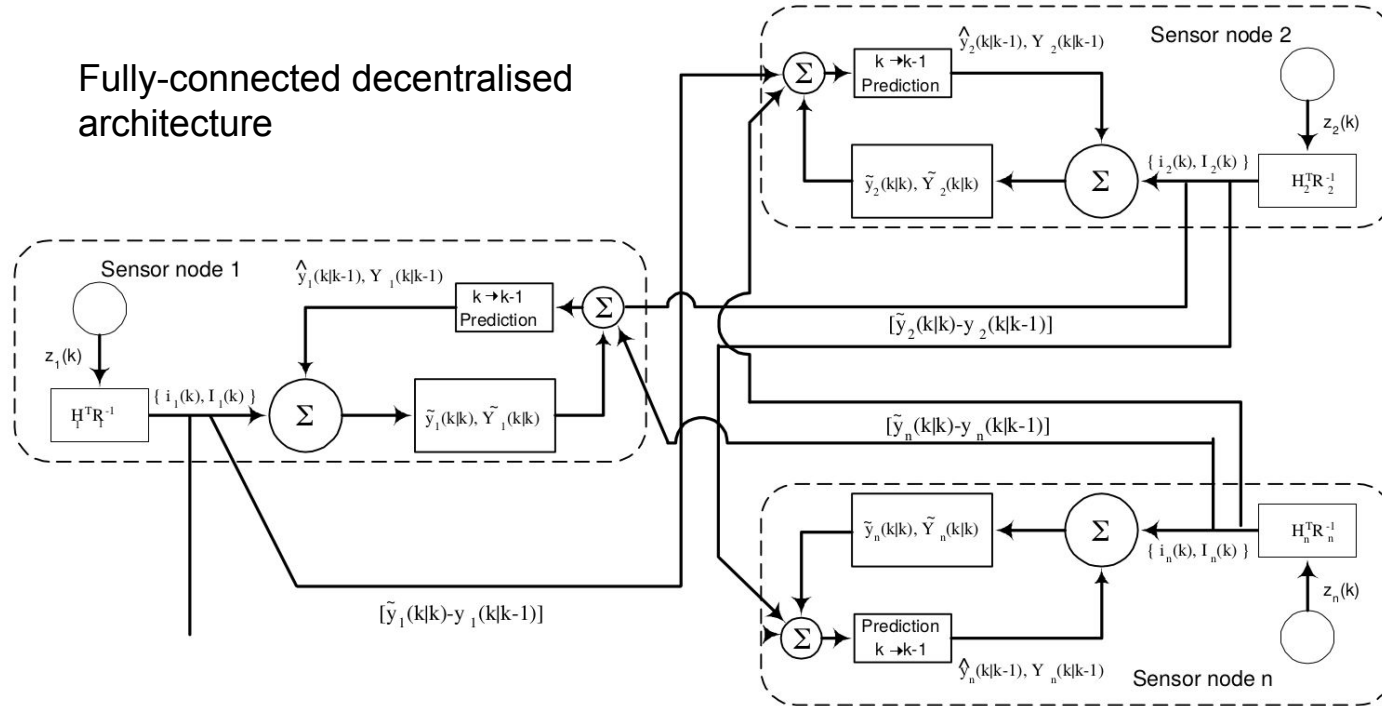


Broadcast medium is a good model of real communication network



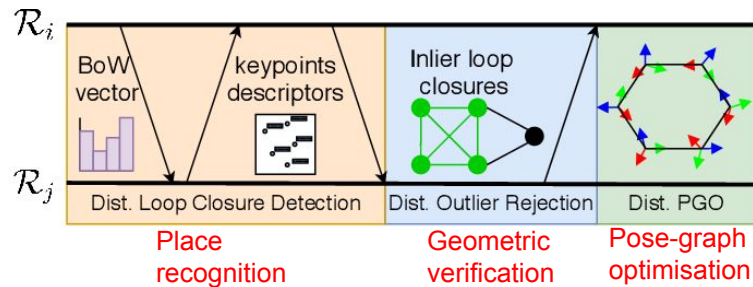
Hybrid broadcast medium and point-to-point architecture

Decentralised Information Filter (early 2000s)

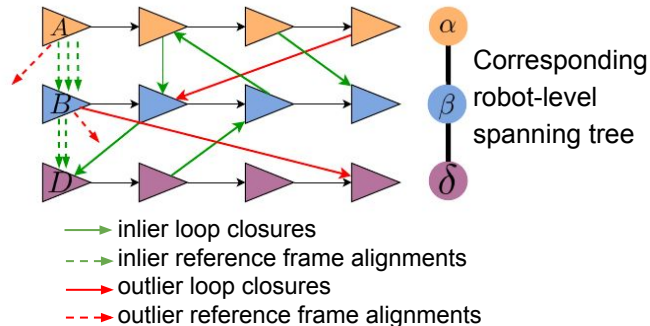


Collaborative SLAM - processes and protocols required

(1) Distributed loop closure detection



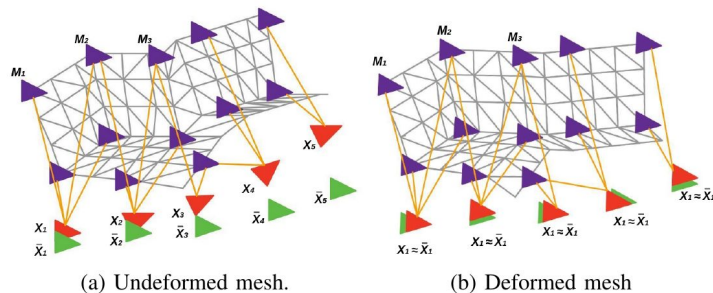
(2) Robust distributed initialisation



(3) Robust distributed pose-graph optimisation

$$\min_{\substack{\mathbf{x}_{\alpha_i} \in \text{SE}(3), \\ \forall \alpha \in \mathcal{R}, \forall i}} \underbrace{\sum_{\alpha \in \mathcal{R}} \sum_{i=1}^{n_{\alpha}-1} r_{\alpha_i}(\mathbf{x}_{\alpha_i}, \mathbf{x}_{\alpha_{i+1}})^2}_{\text{odometry}} + \underbrace{\sum_{(\alpha_i, \beta_j) \in L} \rho(r_{\beta_i}^{\alpha_i}(\mathbf{x}_{\alpha_i}, \mathbf{x}_{\beta_j}))}_{\text{loop closures}},$$

(4) Local mesh optimisation

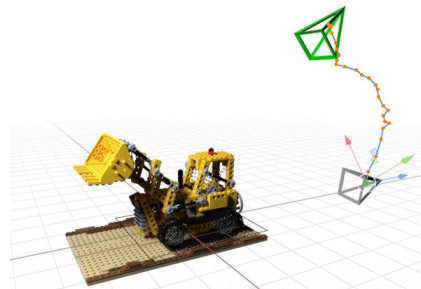
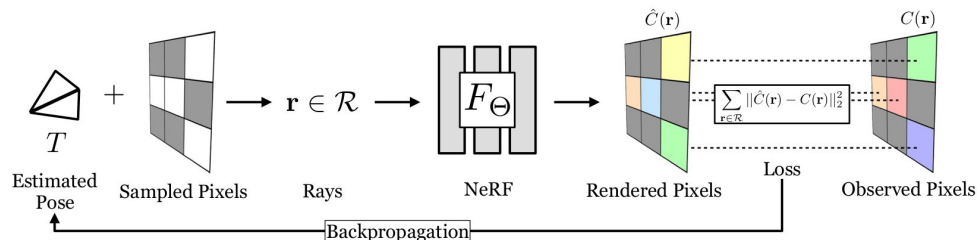


Towards Object-SLAM with implicit shapes

Other architectures

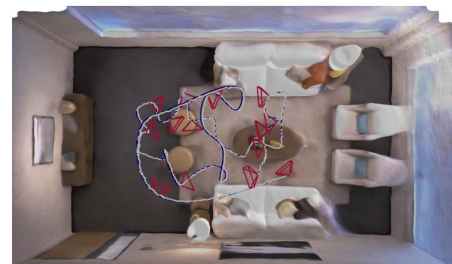
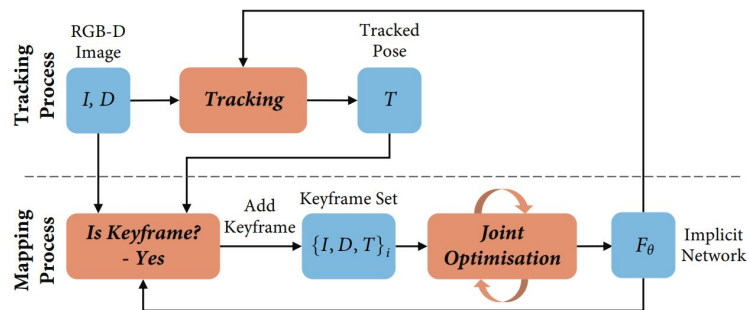
iNeRF

Training 12 hours
Optimisation 30 sec
One global MLP



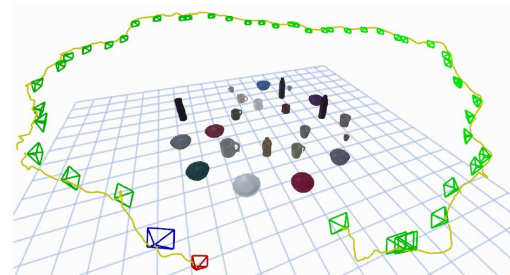
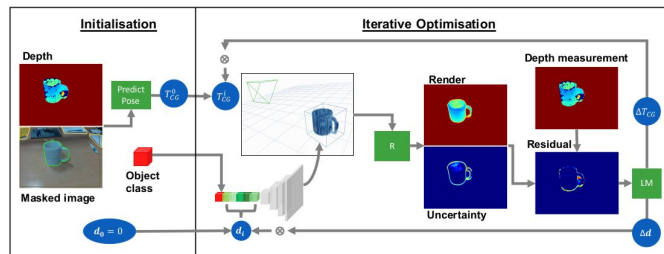
iMAP

Tracking 10Hz
Optimisation 2Hz
One global MLP



NodeSLAM

Tracking ??
Optimisation ??
Auto-decoder with variable latents
MaskRCNN input



CNN + MLP

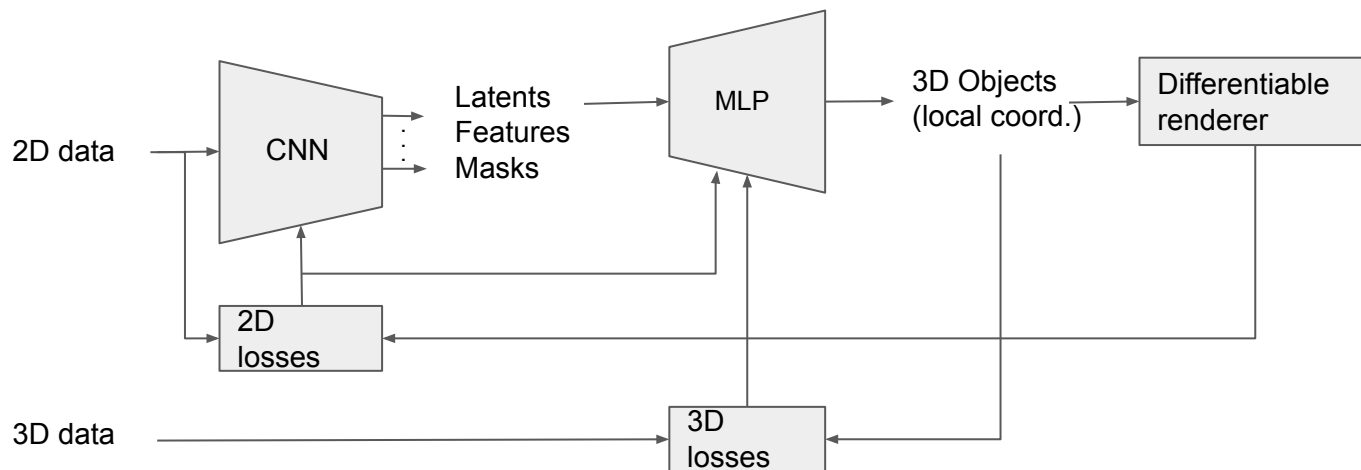
CNN is local (position invariant).

MLP / auto-decoder is global.

Difficulty with modeling the background.

Can they provide a scalable real-time solution?

Generates objects in normalised object coordinates.



Neural Scene Graphs for Dynamic Scenes

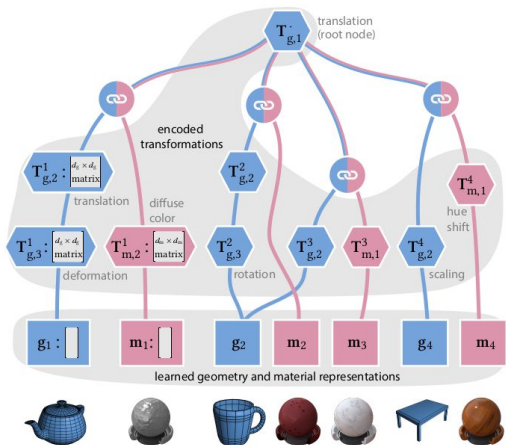
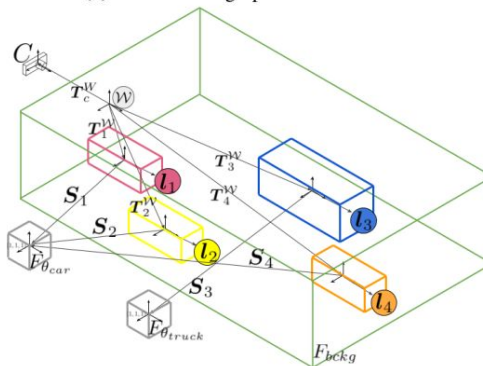


Fig. 1. Our scene graphs consist of leaf nodes (drawn as rectangles) that contain learned vectors storing geometry (blue) and materials (pink), and interior nodes that hold transformations (hexagons), and combine them into object instances (blue-pink circles). Symbols and indices are explained in Section 4 and 5. The geometry and material vectors, as well as encoding

(a) Neural scene graph in isometric view.



(b) Neural scene graph from the ego-vehicle view.

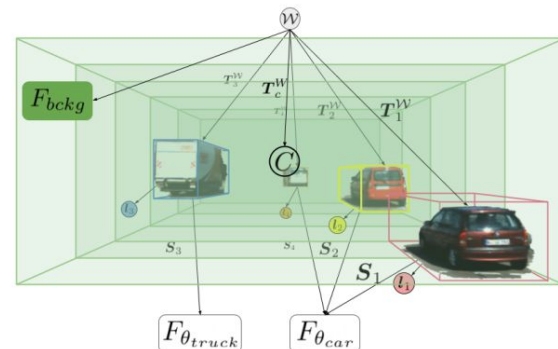
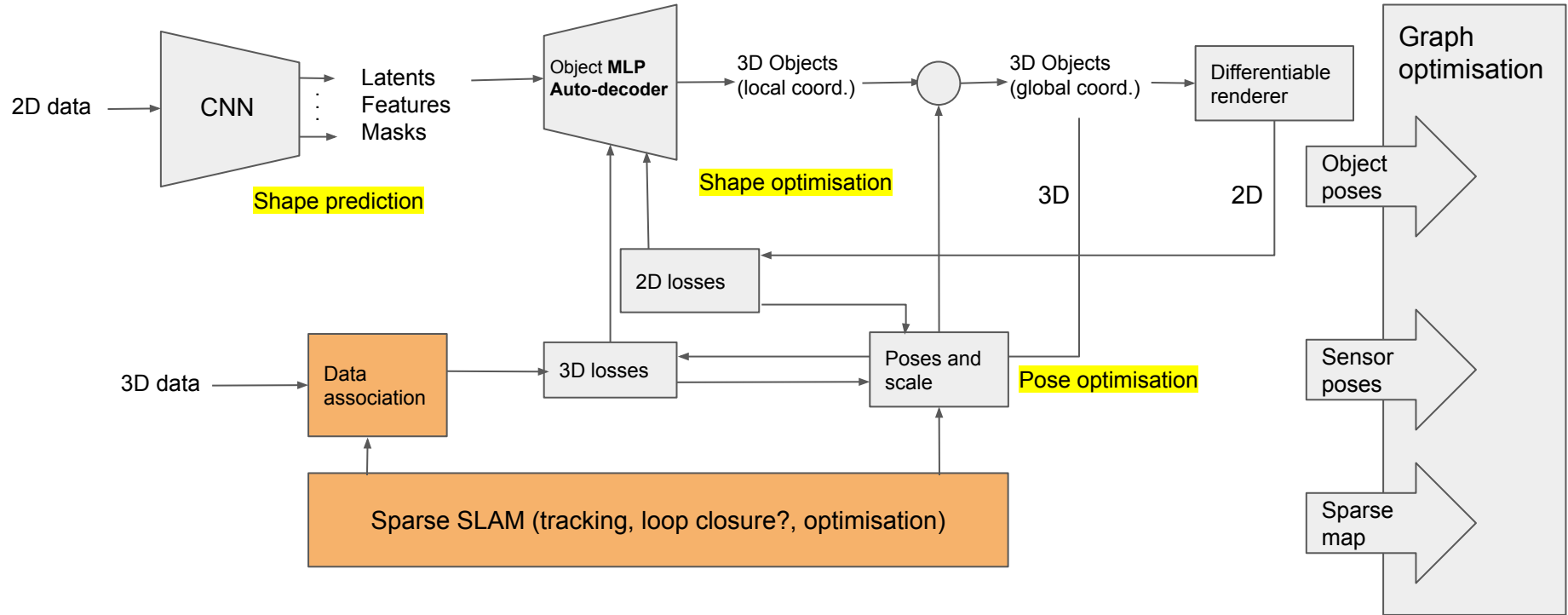
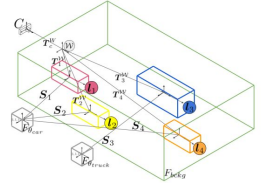
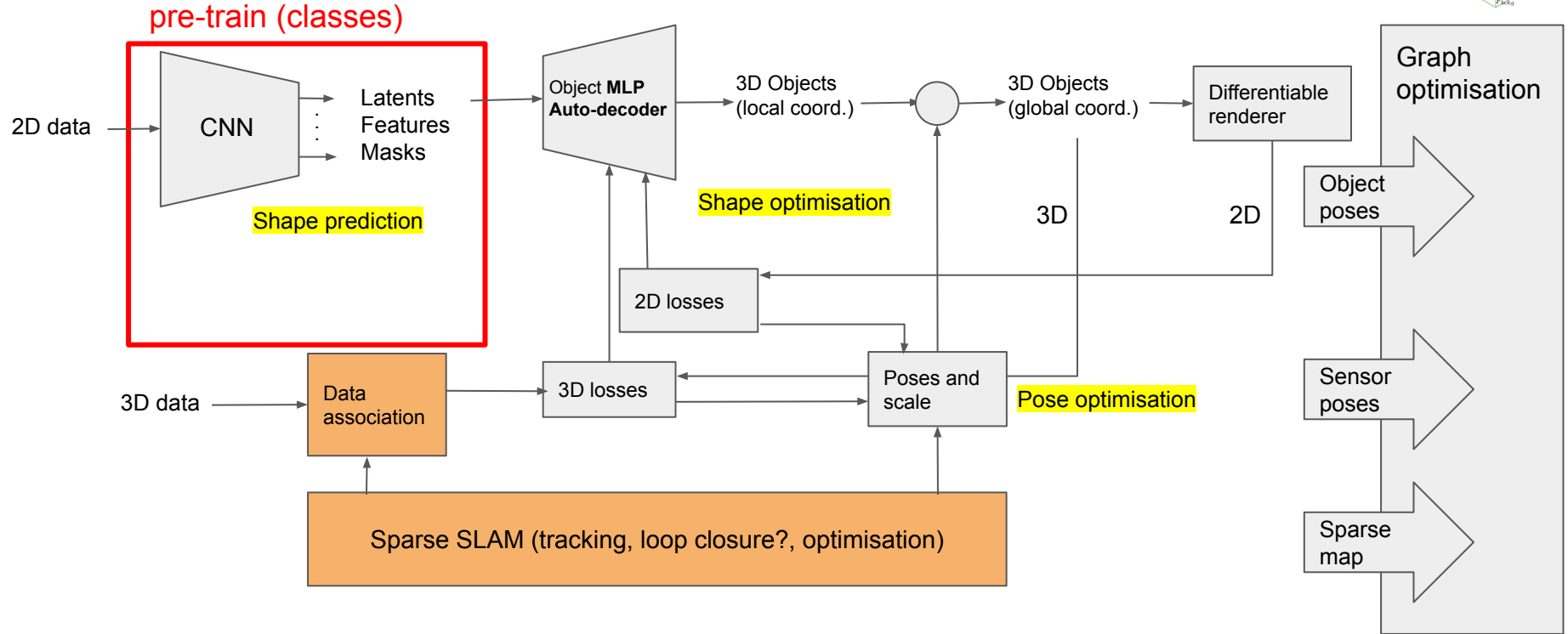
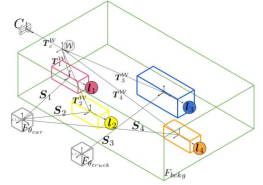


Figure 1: (a) Three dimensional and (b) projected view of the same scene graph S_i . The nodes are visualized as boxes with their local cartesian coordinate axis. Edges with transformations and scaling are visualized by arrows with their transformation and scaling matrix, T_o^w and S_o . The corresponding latent descriptors are denoted by l_o and the representation nodes for the unit-scaled bounding boxes by F_θ . No transformation is assigned to the background, which is already aligned with $\mathcal{W}$.

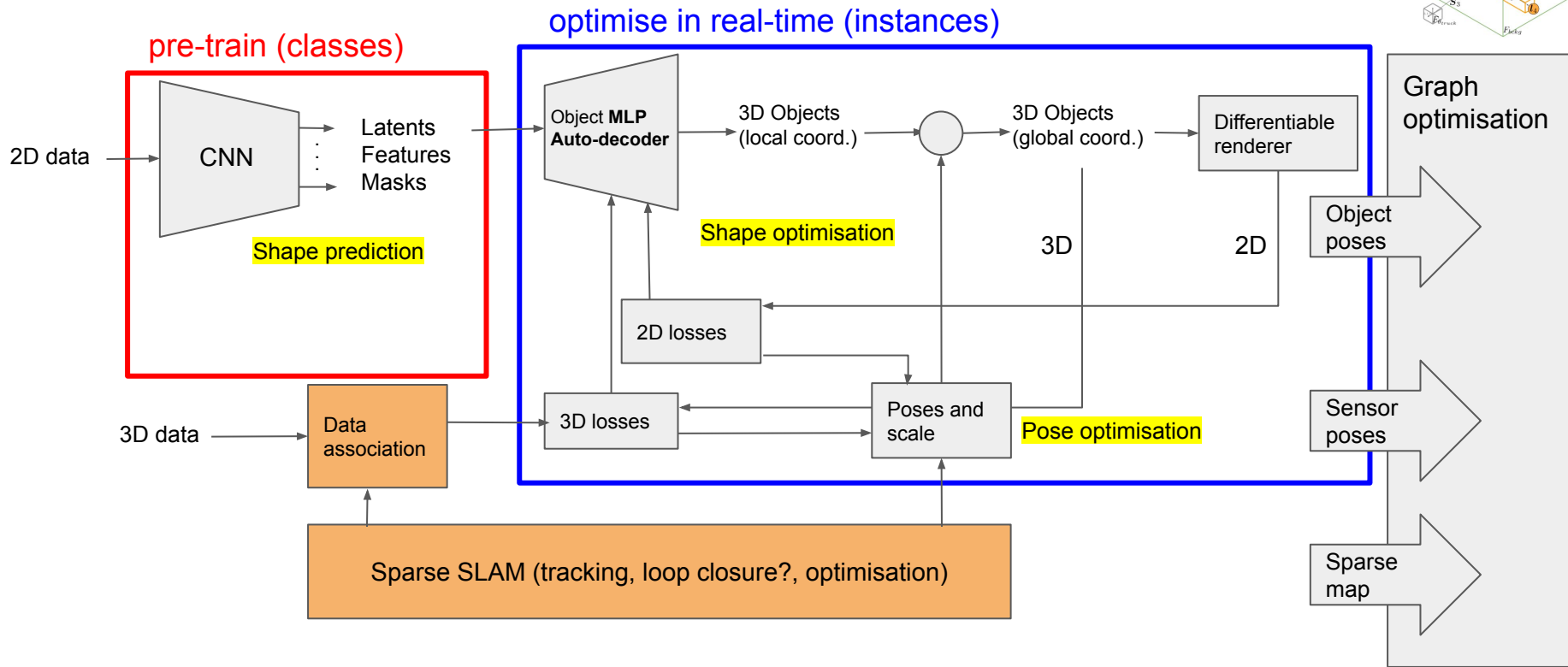
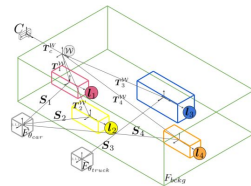
SLAM assessed deep shapes, multiple Networks



SLAM assessed deep shapes, multiple Networks

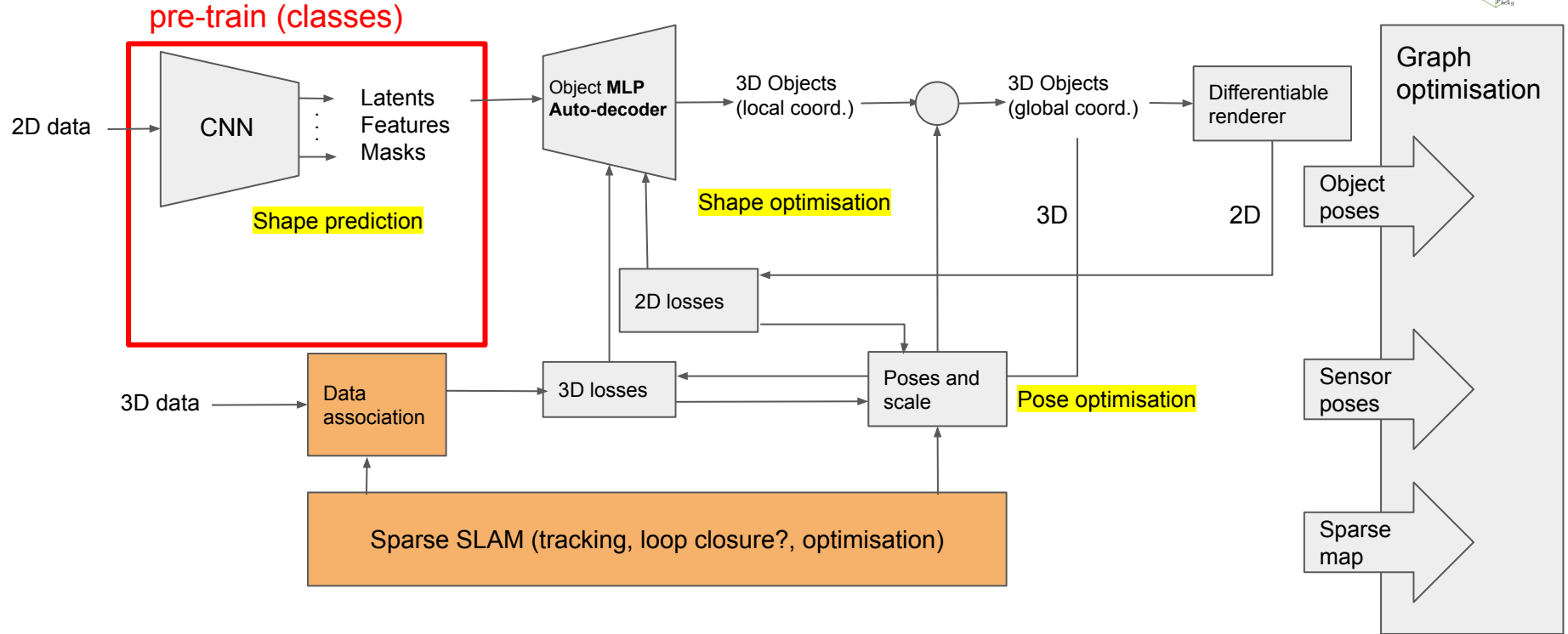
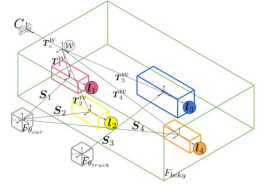


SLAM assessed deep shapes, multiple Networks

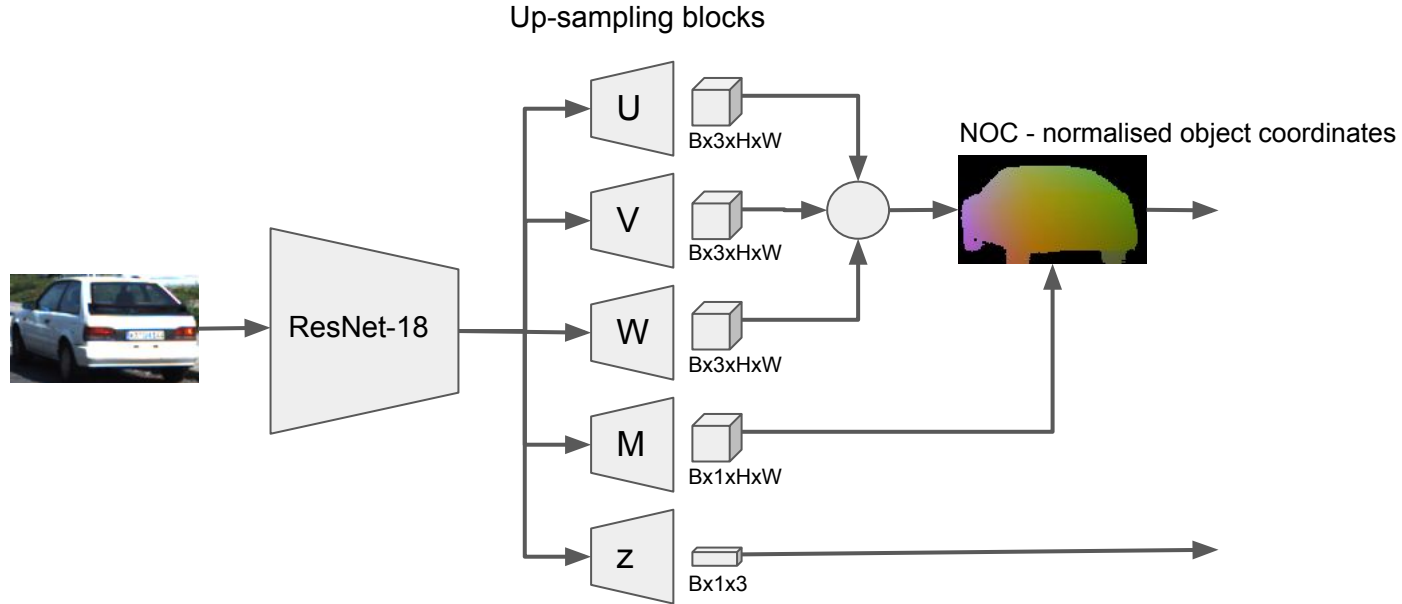


Implementations

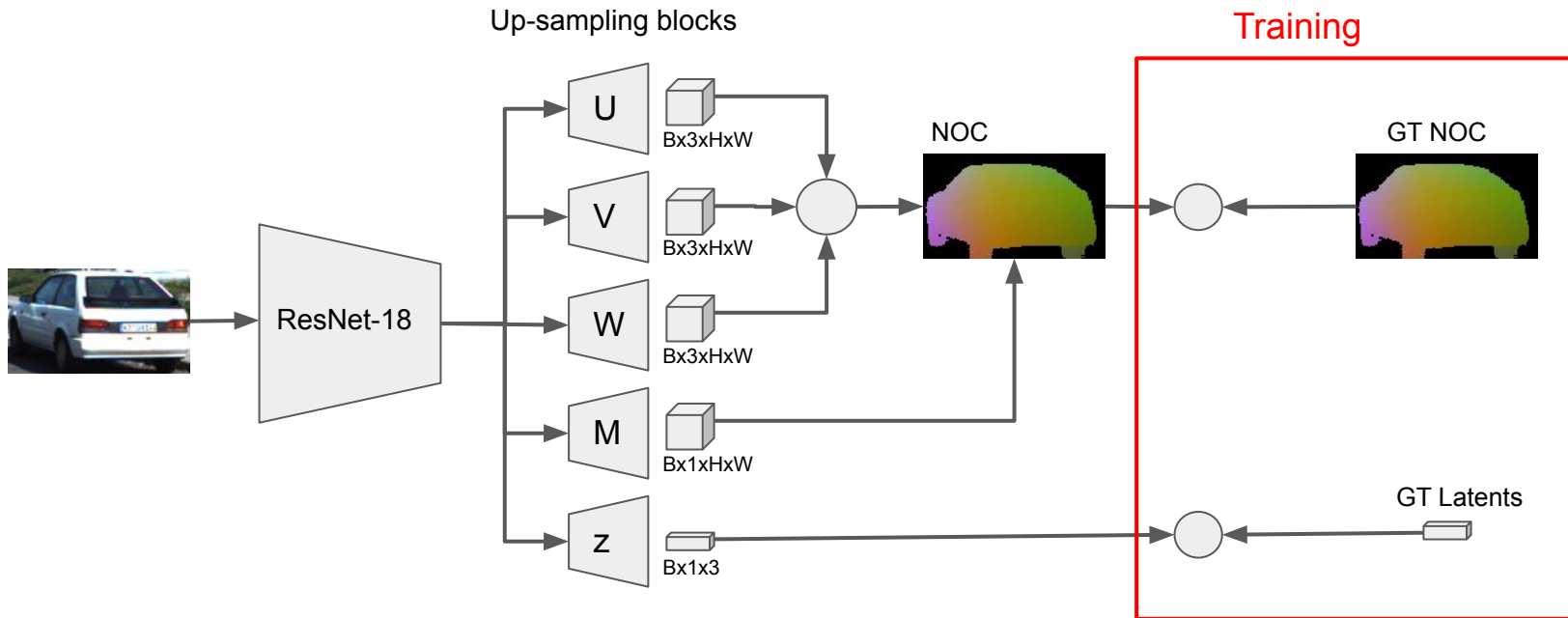
SLAM assessed deep shapes, multiple Networks



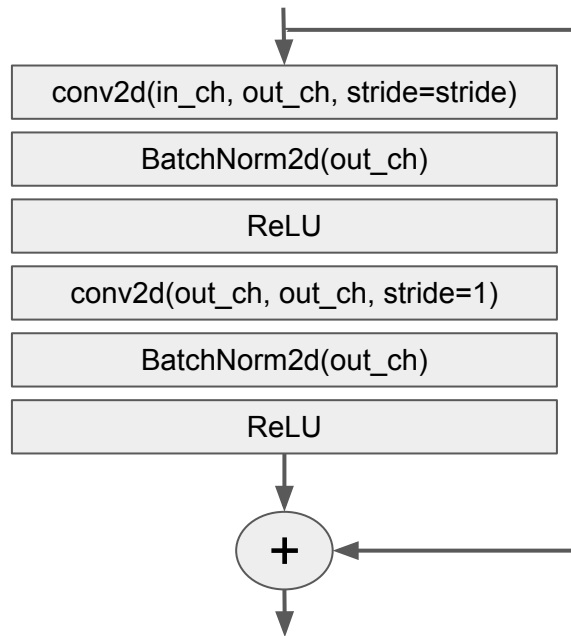
Predictions using pre-trained CNN



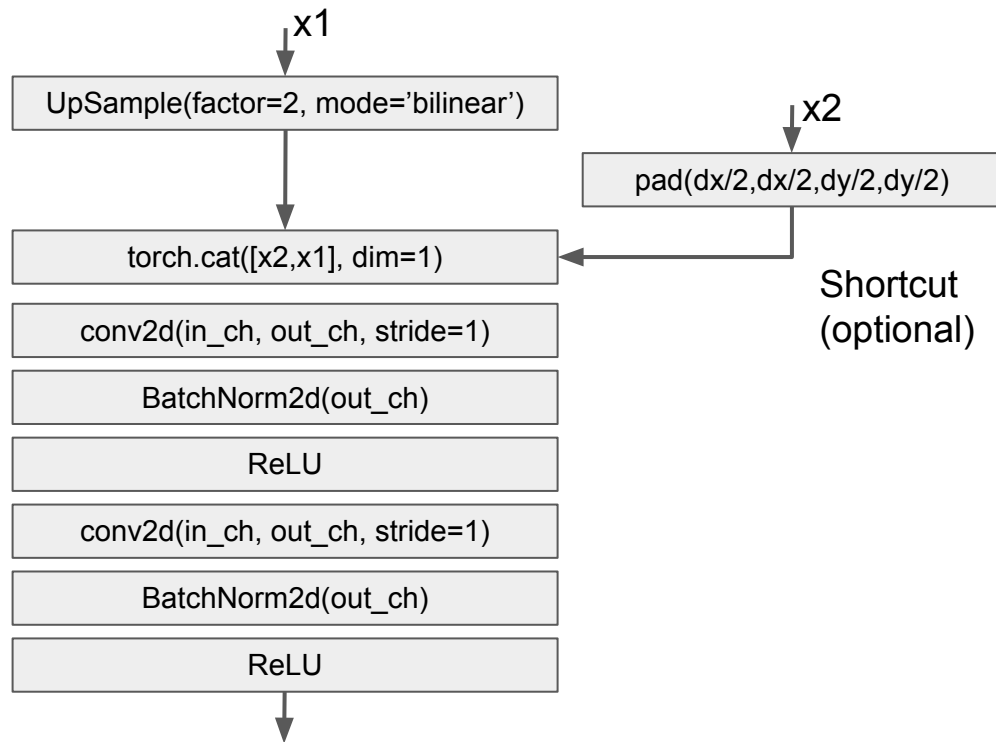
Predictions using pre-trained CNN



BasicBlock (out_ch, stride)
forward (x)



Up (in_ch, out_ch)
forward (x1,x2)



ResNet-18

x: 480x640x3

conv2d(3,64,kernel=7,stroke=2,padding=3)

x1: 240x320x64

BatchNorm2d(64)

ReLU

max_pooling2d(kernel=3,stroke=2,padding=1)

x2: 120x160x64

BasicBlock(64)

BasicBlock(64)

BasicBlock(128, stroke=2)

BasicBlock(128)

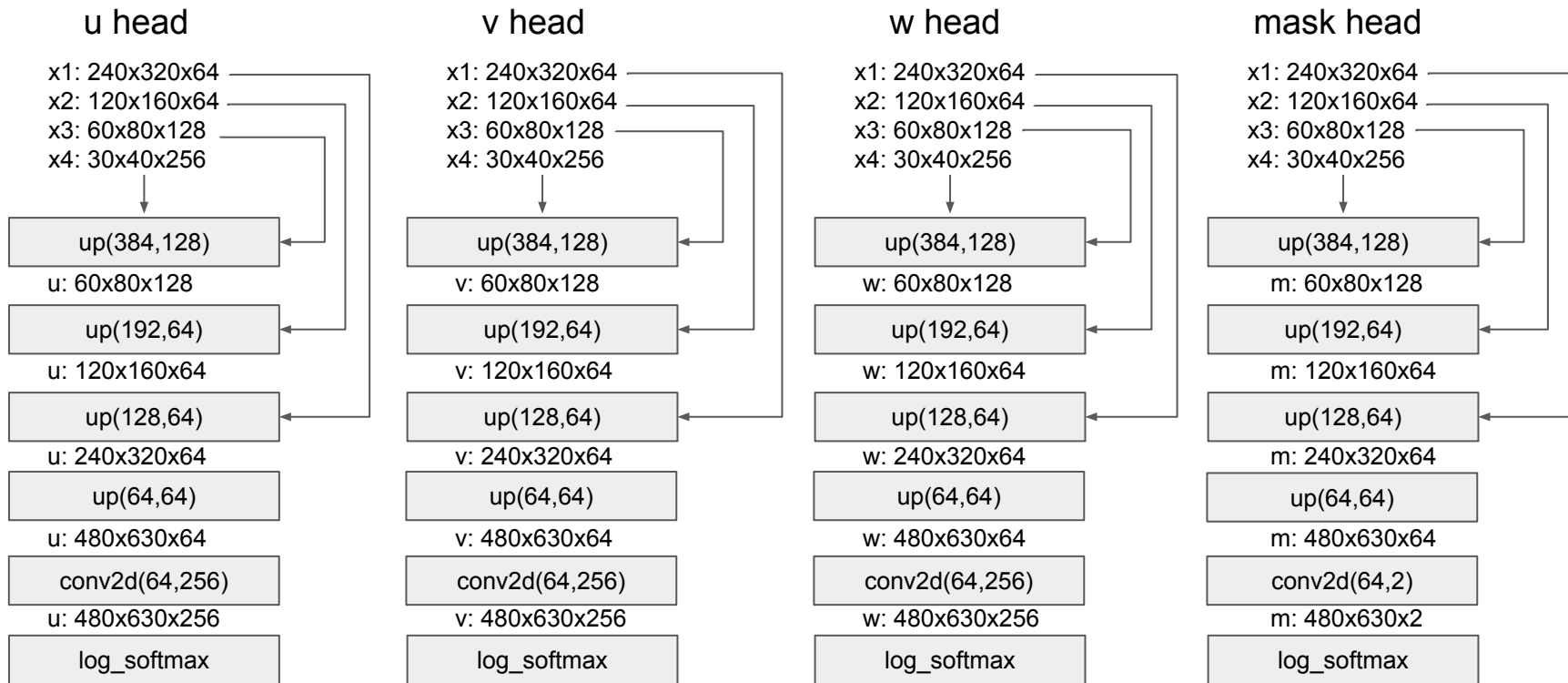
x3: 60x80x128

BasicBlock(256, stroke=2)

BasicBlock(256)

x4: 30x40x256

Prediction heads



Prediction heads

Latent head

x4: 30x40x256

conv2d(256,3)

sigmoid (optional)

mean

z: 1x3

u head

x1: 240x320x64
x2: 120x160x64
x3: 60x80x128
x4: 30x40x256

up(384,128)

u: 60x80x128

up(192,64)

u: 120x160x64

up(128,64)

u: 240x320x64

up(64,64)

u: 480x630x64

conv2d(64,256)

u: 480x630x256

log_softmax

NOC

v head

x1: 240x320x64
x2: 120x160x64
x3: 60x80x128
x4: 30x40x256

up(384,128)

v: 60x80x128

up(192,64)

v: 120x160x64

up(128,64)

v: 240x320x64

up(64,64)

v: 480x630x64

conv2d(64,256)

v: 480x630x256

log_softmax

w head

x1: 240x320x64
x2: 120x160x64
x3: 60x80x128
x4: 30x40x256

up(384,128)

w: 60x80x128

up(192,64)

w: 120x160x64

up(128,64)

w: 240x320x64

up(64,64)

w: 480x630x64

conv2d(64,256)

w: 480x630x256

log_softmax

mask head

x1: 240x320x64
x2: 120x160x64
x3: 60x80x128
x4: 30x40x256

up(384,128)

m: 60x80x128

up(192,64)

m: 120x160x64

up(128,64)

m: 240x320x64

up(64,64)

m: 480x630x64

conv2d(64,2)

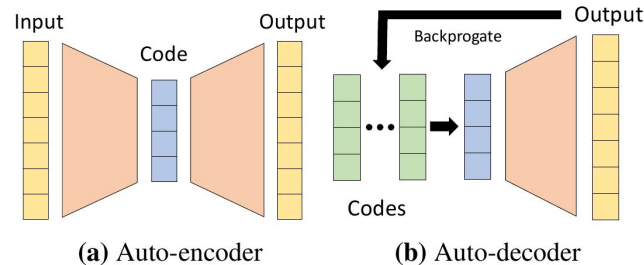
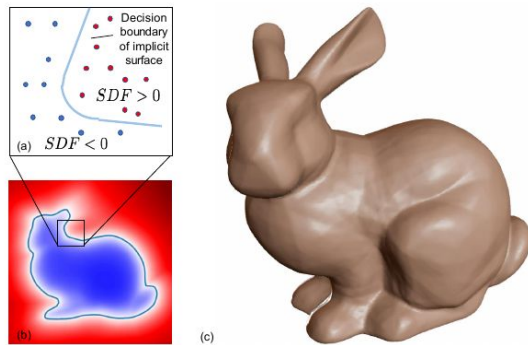
m: 480x630x2

log_softmax

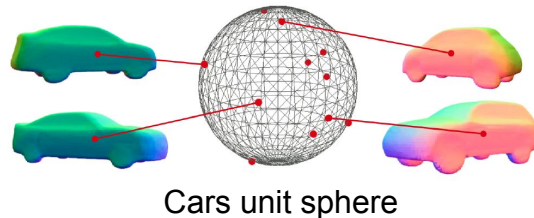
Prob. of
occupancy

Facebook's work on DeepSDF

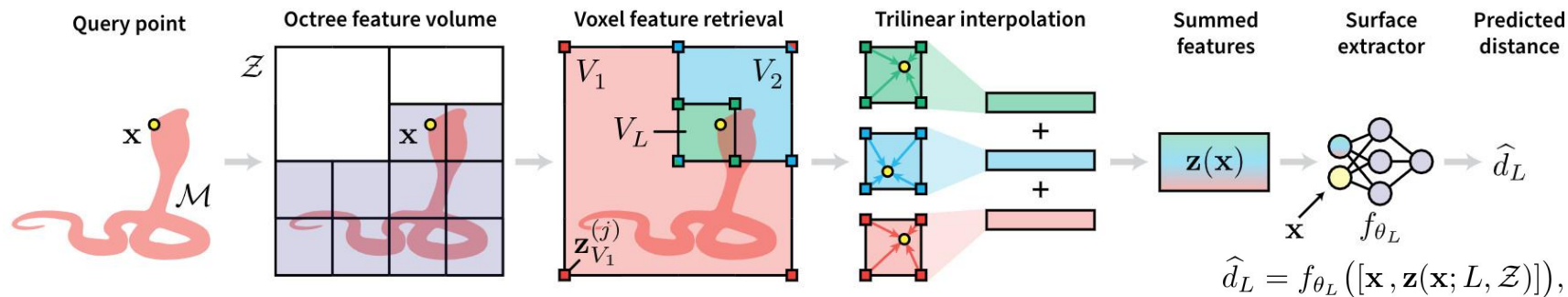
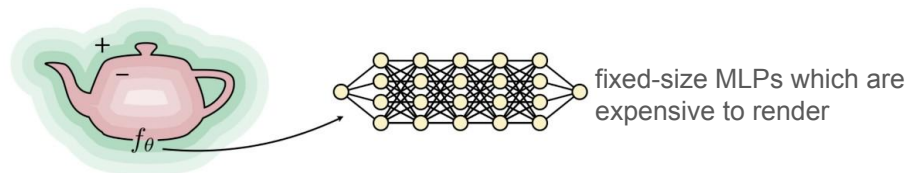
- A signed distance function SDF:
 - A continuous function.
 - $SDF(\mathbf{x}) = s : \mathbf{x} \in \mathbb{R}^3, s \in \mathbb{R}$.
- The idea is to directly regress the SDF from point samples using DNN.
- Train parameters of a multi-layer fully-connected neural network to make its implicit function a good approximation of a given SDF.
 - $f_{\theta}(\mathbf{x}) \approx SDF(\mathbf{x}), \forall \mathbf{x} \in \Omega$.
 - $\mathcal{L}(f_{\theta}(\mathbf{x}), s) = | \text{clamp}(f_{\theta}(\mathbf{x}), \delta) - \text{clamp}(s, \delta) |$,
 - $\text{clamp}(x, \delta) := \min(\delta, \max(-\delta, x))$ - controls details and speed
- Accurate normals can be analytically computed by calculating the spatial derivative $n_i = \frac{\partial f(x_i; \mathbf{z})}{\partial x_i}$ via back-propagation.
- Latent variables can encode shape variations (best viewed as spherical projections of 3D latent vectors).



Auto-decoder: Less encoder-learning



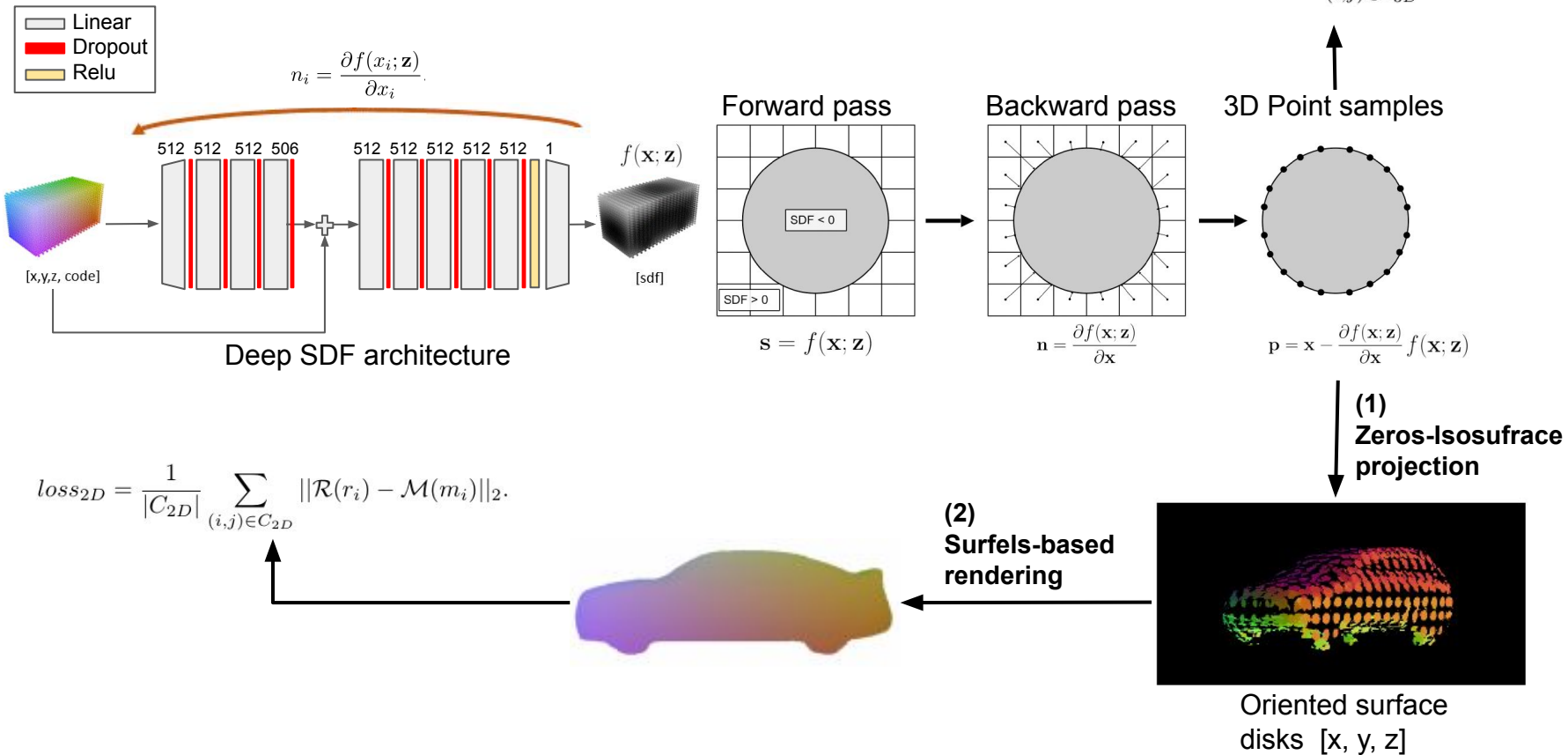
Nvidia's work on Neural SDFs



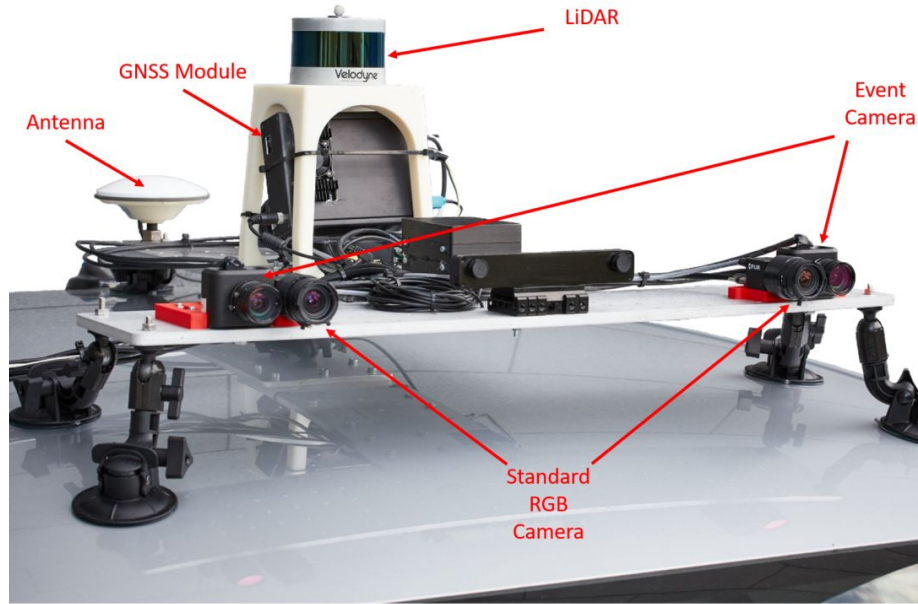
We encode shapes using a sparse voxel octree (SVO) based feature volume, where the levels of the tree corresponds to LOD. This lets us use a very small MLP, which accelerates rendering by 2-3 orders of magnitudes.

$$J(\theta, \mathcal{Z}) = \mathbb{E}_{\mathbf{x}, d} \sum_{L=1}^{L_{\max}} \|f_{\theta_L}([\mathbf{x}, \mathbf{z}(\mathbf{x}; L, \mathcal{Z})]) - d\|^2.$$

An implementation of a Differentiable SDF renderer



KITTI3D dataset

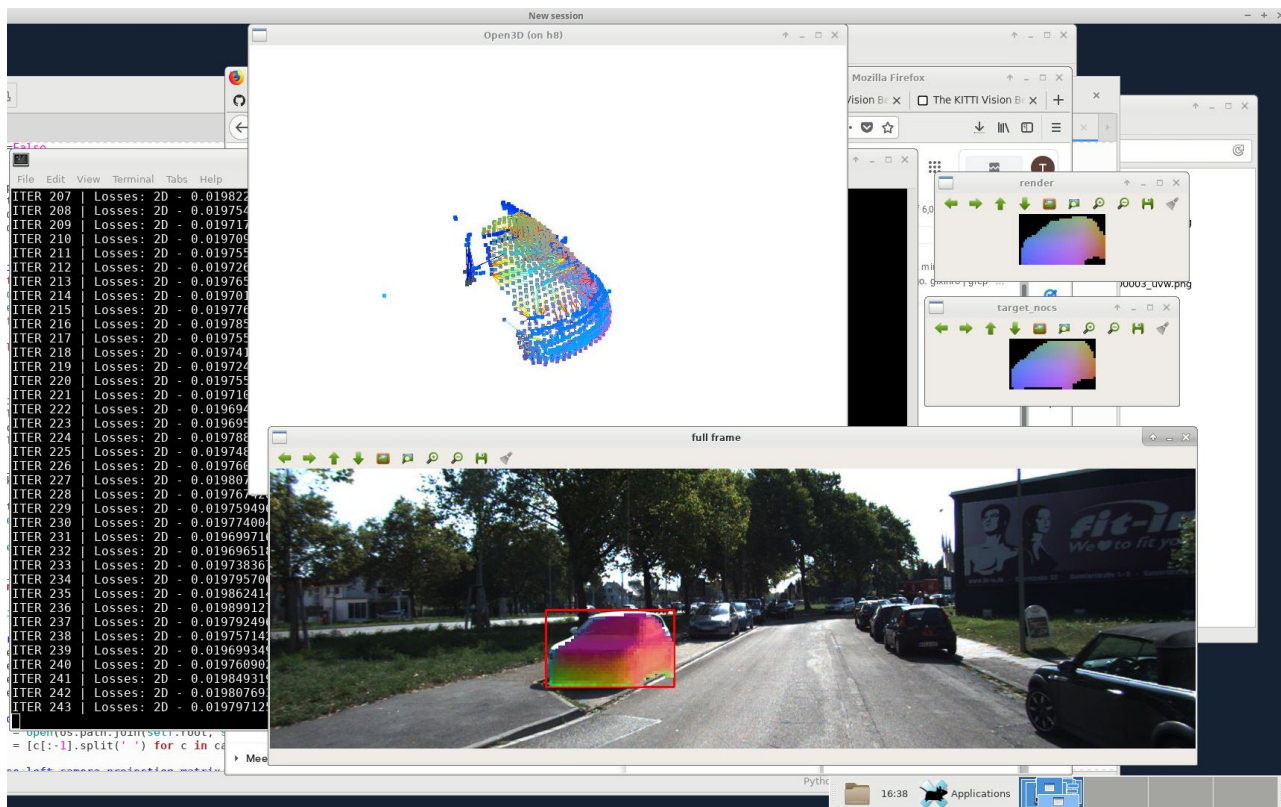


Easy: Min. bounding box height: 40 Px, Max. occlusion level: Fully visible, Max. truncation: 15 %.

Moderate: Min. bounding box height: 25 Px, Max. occlusion level: Partly occluded, Max. truncation: 30 %.

Hard: Min. bounding box height: 25 Px, Max. occlusion level: Difficult to see, Max. truncation: 50 %.

Image + LiDAR (64 lines) - no motion



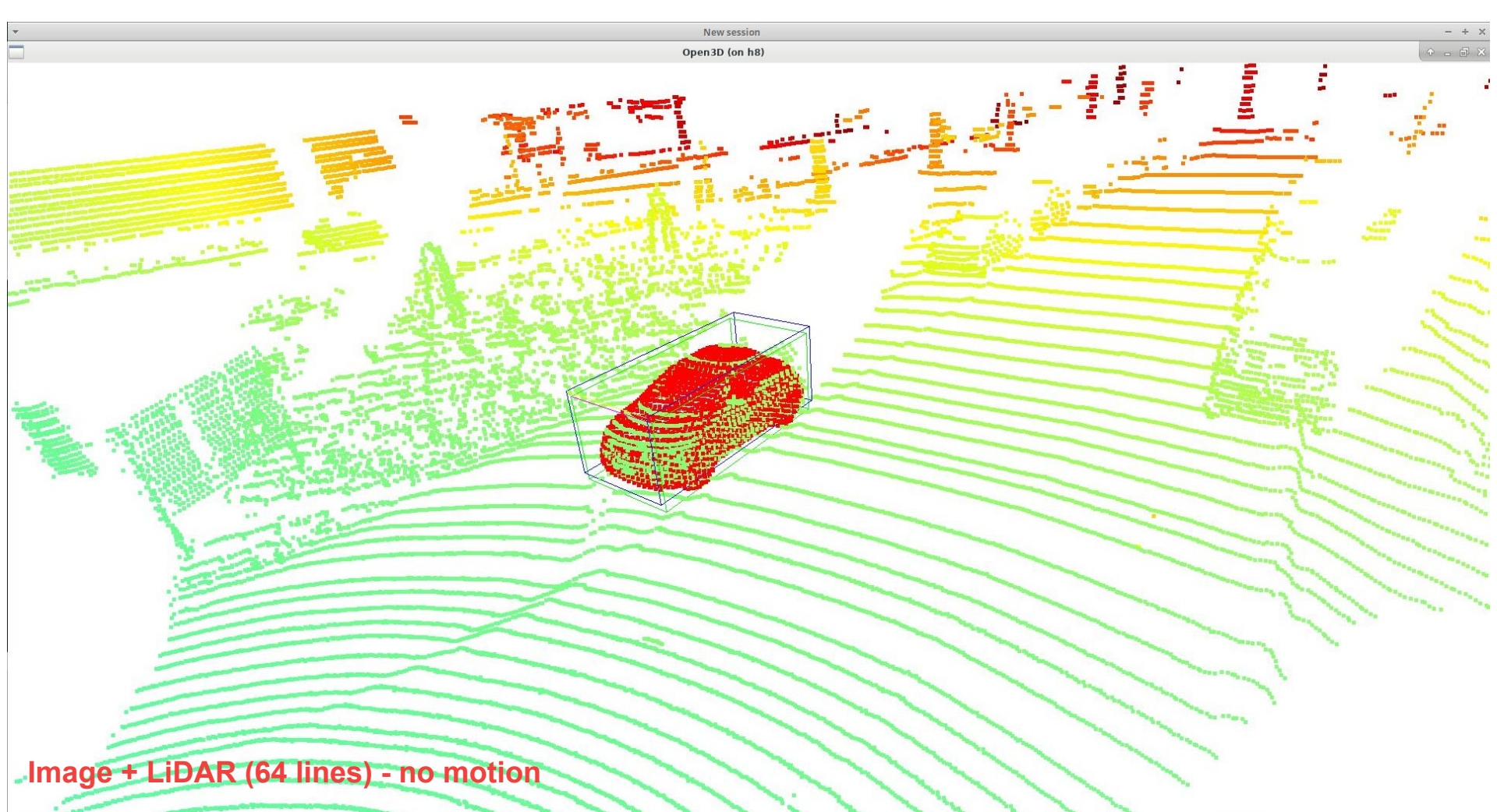
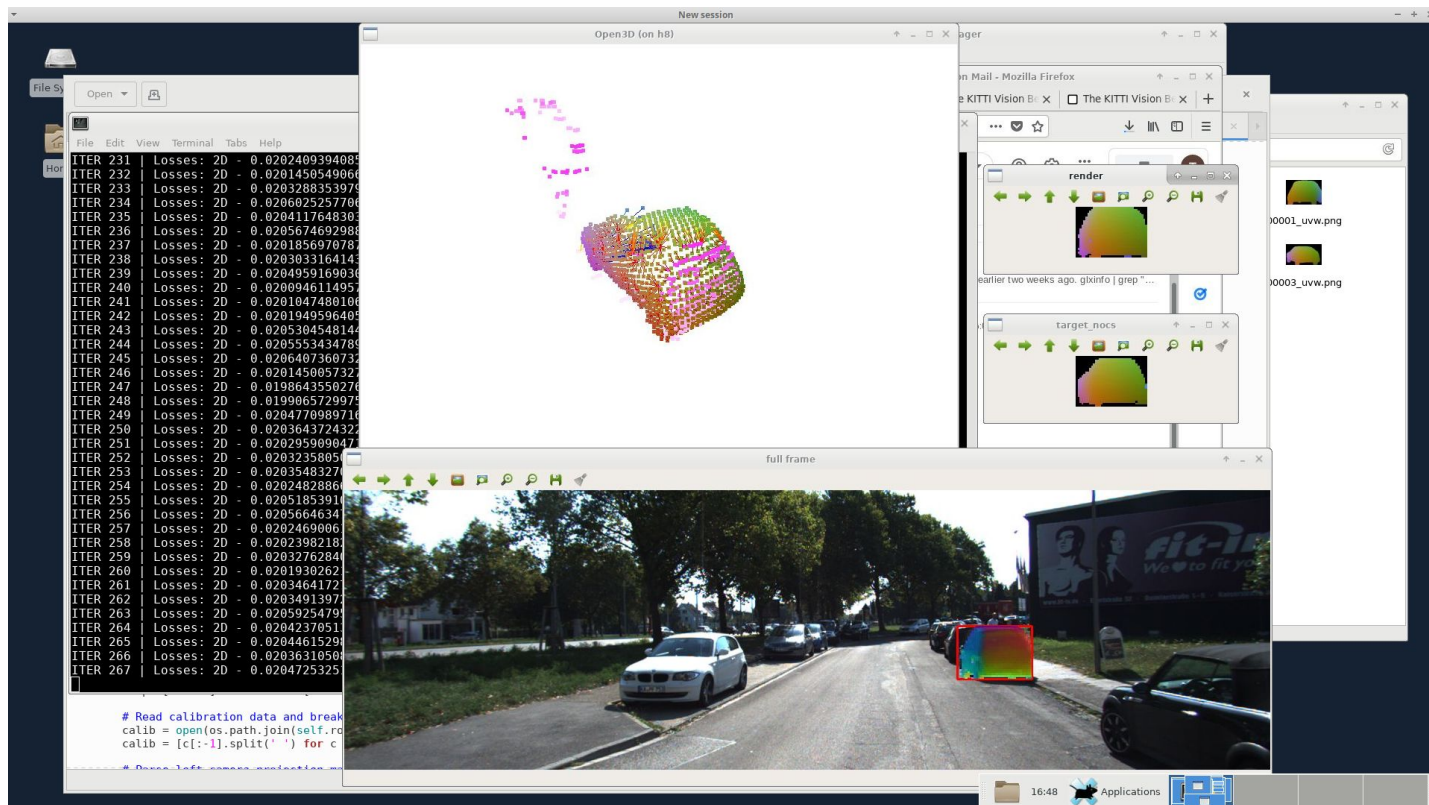


Image + LiDAR (64 lines) - no motion

Image + LiDAR (64 lines) - no motion



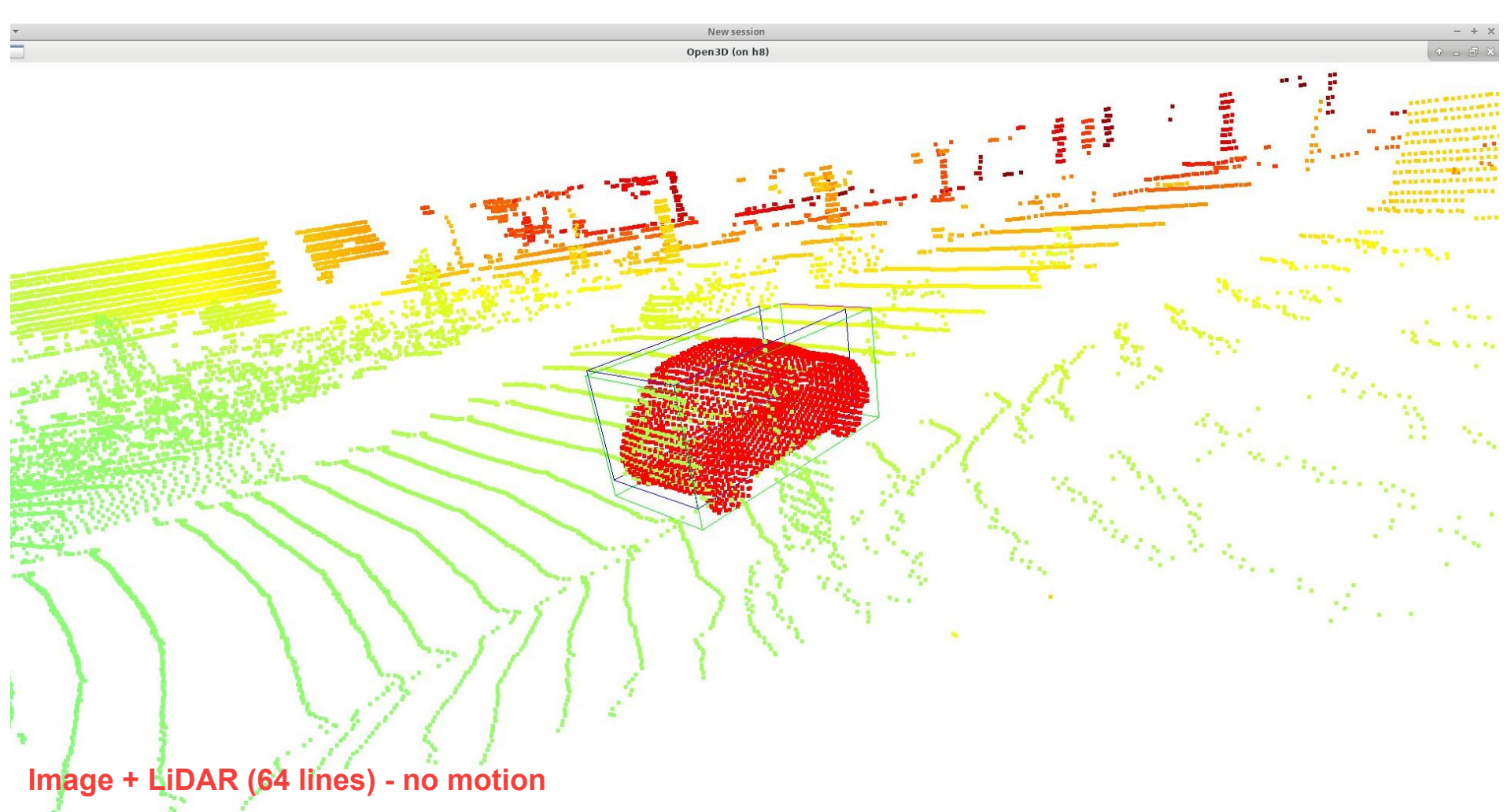
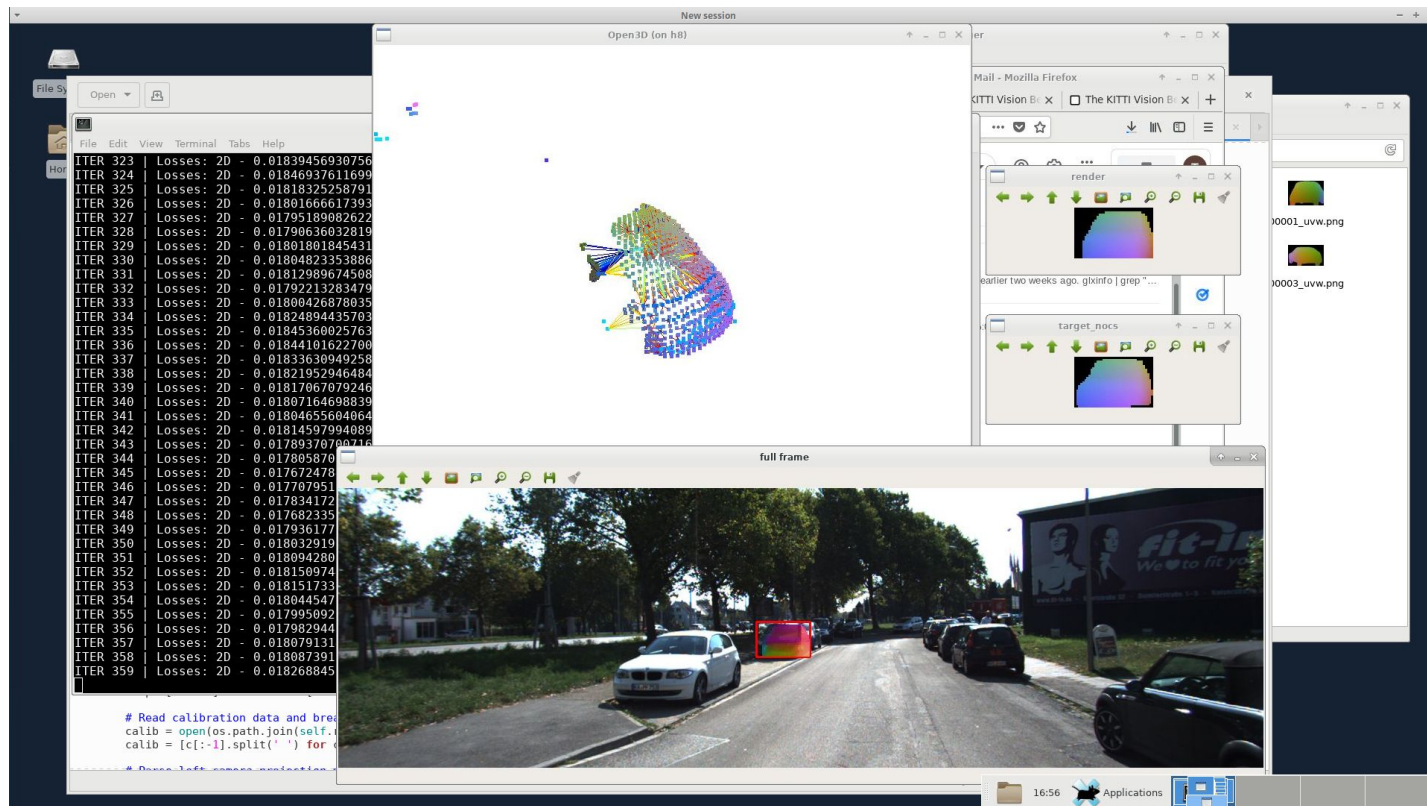
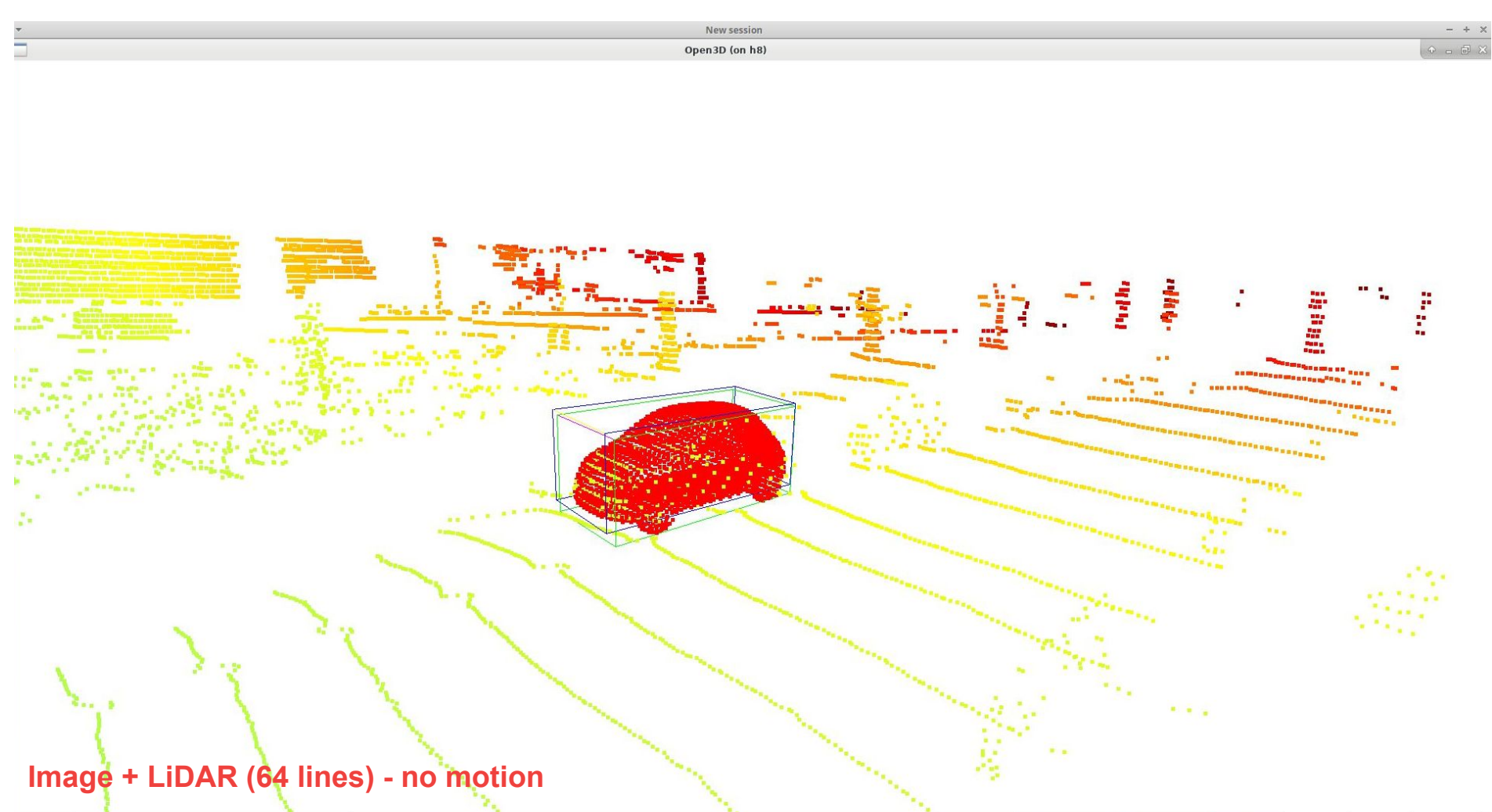
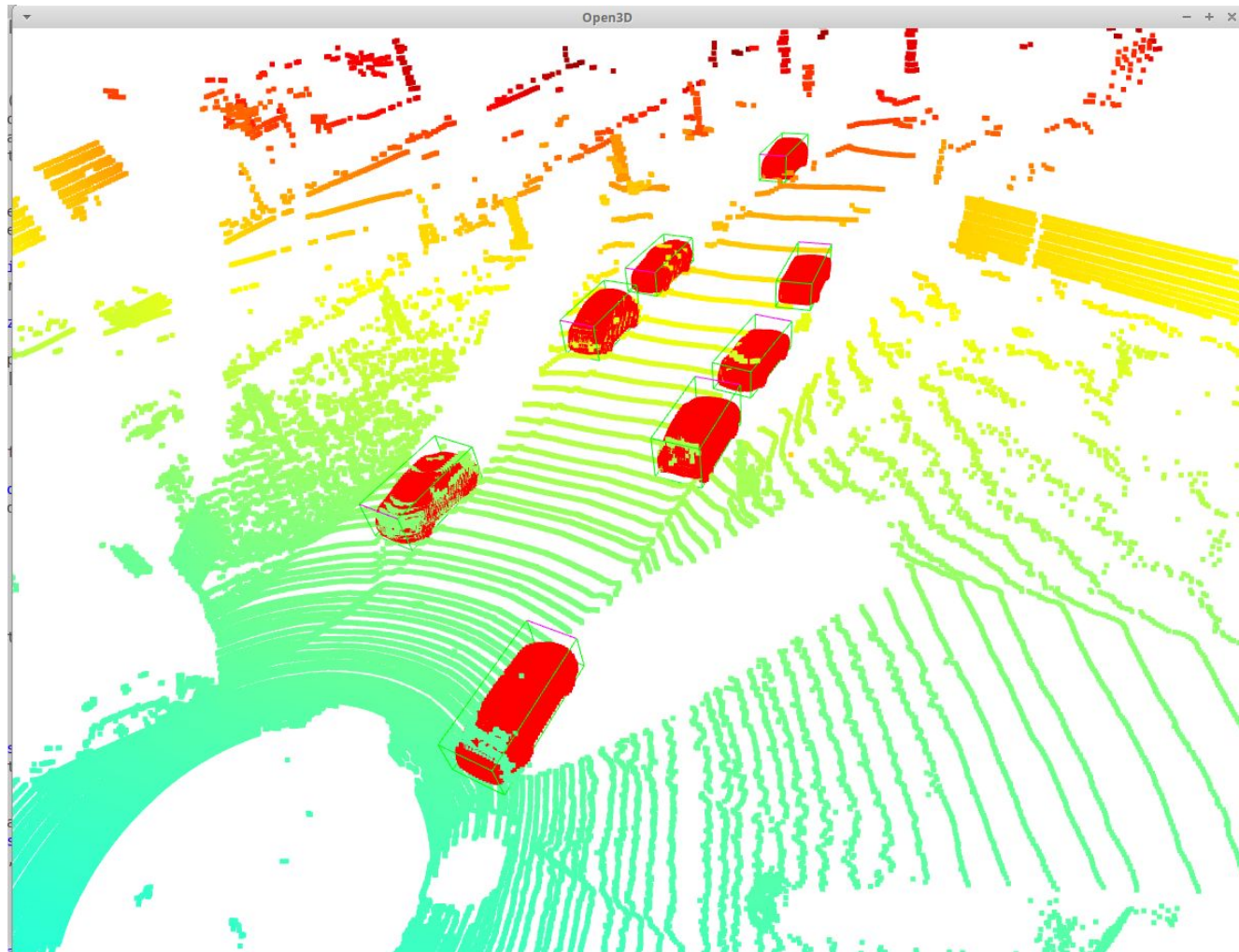


Image + LiDAR (64 lines) - no motion







Whats next ?

- Finish the basic demo.
- Different objective functions to improve silhouette optimisation.
- Automate object detection and cropping within the same network.
- NeRF instead of Deep SDF? What other representations?
- Work with moderate and hard cases (limits; number of points, incomplete shapes).
- Include camera/LiDAR motion
 - Data association (tracking).
 - Incremental update (scale? pose? shape?)
 - Loop closure (remove duplicate objects, repair drift).